

Workflow Pipeline for an Analytical Civic AI Product

A Pipeline Proposal for Civic AI That Evalu- ates Budget Decision Quality, Identifies Hid- den Budget Assumptions, and Distinguishes Budget Evidence from Rhetoric

Dan Swart, CPA (ret)

June 2, 2026

Table of contents

.....	3
.....	3
.....	3
.....	3
.....	3
.....	3
.....	3
Purpose	3
Product framing	4
Current toy RAG compared with proposed analytical AI	5
Recommended funder-facing pipeline	9
Phase 1: Source acquisition and document registry	9
Phase 2: Text extraction, table extraction, and document parsing	10

Phase 4: Validation and reconciliation	13
Phase 5: Analytical database layer	14
Phase 6: Claim, evidence, and assumption extraction	15
Phase 7: Evidence-quality scoring	16
Phase 8: Longitudinal accountability layer	18
Marketplace-scale challenges beyond the current demo	19
PDF-to-data reliability	19
Municipal data is inconsistent	20
The system must distinguish facts from derived calculations ...	20
Methodology can hide premature conclusions	21
Citations must apply to numbers, not just prose	21
Human review remains necessary	22
The buyer may not be the natural beneficiary	23
Proposed product architecture	24
Layer 1: Evidence collection	24
Layer 2: Retrieval layer	24
Layer 3: Structured data layer	25
Layer 4: Analytical tool layer	26
Layer 5: Argument and assumption layer	26
Layer 6: AI answer layer	27
Layer 7: Governance and quality assurance layer	28
Suggested funder-facing development stages	28
Stage 1: Retrieval MVP	28
Stage 2: Structured-data MVP	29
Stage 3: Analytical MVP	30
Stage 4: Accountability MVP	31
Stage 5: Marketplace pilot	32
Funder narrative	33
Blunt summary for funders	34
Appendix: Practical implementation notes	35

Likely transition from local proof of concept to enterprise-ready stack	35
Recommended platform transition	36
Recommended enterprise architecture	44

Filename:

analytical_ai_workflow_pipeline_chat.qmd

This is NOT a ‘you-can’t-do-it’ report. It is a technical assessment of the challenges of building out a fully functioning analytical AI engine as I see it right now. Every new product has challenges. If technical challenges cannot be overcome at reasonable cost, consider having AI take it as far as possible, then prepare downloadable files that a human can subject to textual and numerical analysis, independent from the AI engine.

See in particular comparison Tables below

Purpose

This document describes a workflow pipeline suitable for presentation to funders. The goal is to explain the difference between a basic

retrieval-based AI demonstration and a serious analytical AI product for municipal transparency, financial analysis, and civic accountability.

The central distinction is this:

A basic RAG chatbot finds and summarizes raw data records.

An analytical civic AI must build a trusted evidence layer, a structured data layer, and an auditable reasoning layer before it can safely produce analysis.

The current proof-of-concept demonstrates that a user can ask questions in plain English and receive source-backed answers from public municipal records. That is useful, and prevalent today, but it is not the same as an analytical product that can reliably calculate growth rates, compare budgets, interpret compensation trends, evaluate assumptions, and connect decisions to later outcomes.

Product framing

The current proof-of-concept answers who, what and where questions such as:

“Where in the council packets was compensation discussed?”

The proposed analytical AI should answer effectiveness questions such as:

“Did the City’s stated justification for compensation increases match the evidence, and did later outcomes support the claim?”

That requires more than retrieval. It requires a system that can:

1. collect municipal documents and data,
2. extract text and tables reliably,
3. convert financial information into validated structured data,
4. preserve citations back to source documents,
5. analyze trends, growth rates, inflation comparisons, and fiscal impacts,
6. identify assumptions, weak evidence, and rhetorical framing,
7. compare claims across years,
8. produce transparent outputs that users can inspect, download, and challenge.

Current toy RAG compared with proposed analytical AI

Area	Current toy product / simulated RAG	Proposed analytical AI product
Core purpose	Retrieve relevant passages from city documents	Analyze claims, numbers, assumptions, trends, trade-offs, and evidence quality
Main user question	“Where is this discussed?”	“Is this claim justified by the evidence?”
Data type	Mostly unstructured text from PDFs	Unstructured text plus validated structured financial, budget, payroll, staffing, and outcome data

Area	Current toy product / simulated RAG	Proposed analytical AI product
Storage	Local files, RDS objects, simulated vector database	Production document store, structured SQL database, vector database, metadata registry, audit logs
Chunking	Break PDFs into text chunks	Chunk text, extract tables, tag claims, identify fiscal years, departments, funds, accounts, motions, recommendations
Embeddings	Used to find semantically similar passages	Used as one part of retrieval, alongside SQL queries, document metadata, fiscal-year filters, and issue-specific taxonomies
Retrieval	Top-k passages returned to the model	Hybrid retrieval: semantic search, keyword search, SQL lookup, metadata filtering, longitudinal linking
Citations	Cite retrieved document excerpts	Cite every number, claim, calcu-

Area	Current toy product / simulated RAG	Proposed analytical AI product
		lation, and interpretation back to source document, table, page, line item, or dataset
Numeric work	Minimal or manual	Automated calculations, growth rates, inflation comparisons, variance analysis, fund-level trends, downloadable data frames
Financial reliability	Not sufficient for formal analysis	Requires extraction validation, reconciliation, schema rules, data dictionaries, and human review checkpoints
Argument analysis	Summary only	Extract claim → evidence → assumption → recommendation → decision → later outcome
Evidence quality	Not evaluated	Flags unsupported assertions, cherry-picked comparisons, vendor framing,

Area		Current toy product / simulated RAG	Proposed analytical AI product
			peer-city claims, and missing methodology
Longitudinal ory	mem-	Limited to retrieved chunks	Tracks claims, promises, forecasts, votes, outcomes, and later actual results across years
User output		Narrative answer with sources	Narrative answer, citations, tables, charts, downloadable datasets, reproducible calculation trail
Risk level		Low to moderate	High, because errors in numbers or interpretation can mislead public decision-making
Marketplace ness	readi-	Demo / proof of concept	Requires production architecture, quality assurance, governance, validation, security, scalability, legal review, and support model

Recommended funder-facing pipeline

Phase 1: Source acquisition and document registry

The first layer is not AI. It is disciplined public-record collection.

Sources may include:

- council agendas,
- minutes,
- agenda packets,
- budget books,
- annual comprehensive financial reports,
- compensation plans,
- compensation exports obtained by public-information request,
- capital improvement plans,
- staff presentations,
- ordinances and resolutions,
- meeting videos and transcripts (Citizen Portal has made a good start of transcribing video content and indexing it. Feerally they seem to be current, but they are an entire year behind in transcribing Schertz data making it uninformative for this use case)

Each source document needs a registry record

Field	Purpose
Document ID	Permanent internal identifier
Source URL	Where the document came from
Collection date	When it was captured
Document type	Budget, packet, minutes, annual report, payroll file, etc.
Meeting date / fiscal year	Time anchor

Field	Purpose
Department / fund / topic	Analytical filtering
Version status	Draft, adopted, amended, replaced
File hash	Detects whether the file changed
Public citation label	Allows source-backed answers

The first major funding challenge is this:

The AI product is only as reliable as the municipal evidence layer underneath it.

Phase 2: Text extraction, table extraction, and document parsing

The current toy RAG can tolerate imperfect text extraction. An analytical product cannot.

For simple retrieval, it may be acceptable if a paragraph is mostly correct. For financial analysis, that is not enough. The system must distinguish:

- a table title,
- row labels,
- column headings,
- fiscal years,
- totals,
- subtotals,
- notes,
- amended amounts,

- proposed amounts,
- adopted amounts,
- actual amounts,
- one-time transfers,
- recurring expenditures.

A serious pipeline should separate extraction into two tracks.

Track	Purpose
Text extraction	Used for claims, recommendations, meeting narrative, and staff explanations
Table extraction	Used for budgets, compensation, expenditures, revenues, staffing, debt, and capital projects

This is one of the largest jumps from the current demonstration.

A retrieved paragraph can be close enough for civic search. A table used for financial analysis must be right.

Phase 3: Structured data normalization

After extraction, the product needs to turn messy municipal records into analysis-ready tables.

For example, a budget table might become:

fiscal_year	fund	department	account	category	amount_type	amount
2020	General Fund	Police	Salaries	Personnel	Actual	12400000
2026	General Fund	Police	Salaries	Personnel	Proposed	18900000

This is the step that turns PDFs into numbers.

The product needs data dictionaries for terms such as:

- actual,
- adopted,
- amended,
- proposed,
- projected,
- recurring,
- one-time,
- transfer,
- personnel,
- operations,
- capital,
- debt service,
- general fund,
- enterprise fund.

Without this layer, the AI may appear analytical while actually comparing mismatched figures.

The expensive part is not the chatbot.

The expensive part is building a trustworthy municipal data model.

Phase 4: Validation and reconciliation

Before numbers are used by the AI, they need automated and human-auditable checks.

Validation check	Why it matters
Do line items sum to department totals?	Detects extraction errors
Do department totals sum to fund totals?	Detects missing rows
Do adopted totals match the published budget summary?	Confirms the parsed table agrees with the official document
Are fiscal years consistently labeled?	Prevents comparing FY2024 adopted to FY2025 proposed by mistake
Are one-time and recurring items separated?	Prevents false claims about structural growth
Are transfers double-counted?	Avoids overstating expenditure growth
Are employee names, job titles, and salaries normalized?	Enables compensation analysis
Are inflation comparisons tied to the correct time period?	Prevents misleading CPI comparisons

This phase should produce a visible audit trail:

```
Source document
  ↓
Extracted table
  ↓
Cleaned table
  ↓
Validation checks
  ↓
Approved analytical dataset
```

Phase 5: Analytical database layer

A serious product should not rely on the language model to perform calculations from raw text.

The model should call tools.

The analytical layer should include:

- SQL database for structured municipal data,
- vector database for unstructured document retrieval,
- document store for original PDFs,
- calculation engine for growth rates, inflation adjustment, variance analysis, and trend detection,
- reproducible R or Python scripts for higher-risk analysis,
- downloadable tables for users.

A simplified architecture is:

```
Public documents and public-information files
  ↓
Document registry
  ↓
```

Text extraction + table extraction



Structured data cleaning



Validation and reconciliation



Analytical database



AI retrieval + SQL + calculation tools



Narrative answer + citations + charts + downloadable data

The marketplace point is important:

The model should not be trusted to invent calculations.
It should retrieve validated data, run defined calculations, and explain the result.

Phase 6: Claim, evidence, and assumption extraction

This is the most distinctive part of the proposed product.

An analytical AI should not merely summarize staff reports. It should separate claims, evidence, assumptions, recommendations, decisions, and later outcomes.

Object	Example
Claim	"The City must increase salaries to remain competitive."
Evidence	"A peer-city comparison shows Schertz below market."

Object	Example
Assumption	"The selected peer cities are appropriate comparators."
Recommendation	"Adopt a 5% across-the-board raise."
Decision	"Council approved the raise."
Later outcome	"Vacancy rates did or did not improve in the following year."

This creates an argument ledger.

That ledger allows the AI to answer accountability questions:

- What evidence was offered?
- Was the evidence internal or external?
- Was the comparator group justified?
- Was the claim repeated in later years?
- Did the promised result occur?
- Did the recommendation match the evidence?
- Was there a less expensive alternative?
- What was not funded because this was funded?

This moves the product from search to governance analysis.

Phase 7: Evidence-quality scoring

The product should not merely repeat official explanations. It should classify the quality of evidence.

Evidence type	Analytical warning
Internal historical data	Usually stronger if definitions are stable
Audited financial data	Stronger, but may lag current decisions
Public-information payroll data	Strong if complete and reconciled
Peer-city comparison	Highly sensitive to comparator selection
Vendor study	Possible conflict of interest
Staff assertion	May be true, but requires support
Anecdote	Weak evidence unless corroborated
“Industry standard” claim	Needs source and definition
Urgency framing	Requires scrutiny, not automatic acceptance
Forecast	Must later be compared to actual results

Even a simple figure such as “47% growth” embeds choices about:

- start year,
- end year,
- inflation measure,
- fund selection,
- inclusion or exclusion of transfers,
- recurring versus one-time costs,
- department mapping,
- whether population growth is relevant,
- whether service levels changed.

A good analytical AI should expose those choices.

Phase 8: Longitudinal accountability layer

This is where the product becomes meaningfully different from most civic AI tools.

The system should track claims over time, measured by EFFECTIVENESS, not by efficiency.

Prior claim	Later test
"This raise will improve retention."	Anyone would know in advance that turnover decreases with rise in pay. Did the action improve effectiveness? By what method or metric?
"This equipment will reduce maintenance costs."	Did maintenance spending decline? By what method or metric?
"This fee increase is temporary."	Was it later reversed? By what method or metric?
"This position is needed because workload increased."	Was workload data ever provided? Did the action improve effectiveness? By what method or metric?
"This project will cost \$X."	What was the final cost?
"This is a one-time expenditure."	Did it become recurring?

This requires linking across:

- staff reports,
- budget books,

- meeting minutes,
- adopted ordinances,
- later budgets,
- payroll data,
- project reports,
- audit documents.

That is not ordinary RAG. It makes longitudinal data available for study of civic effectiveness.

Marketplace-scale challenges beyond the current demo

PDF-to-data reliability

One of the biggest technical challenge is converting PDFs into accurate structured data.

Retrieval can tolerate imperfect extraction.
Financial analysis cannot.

If a chatbot misses a paragraph, the answer may be incomplete. If an analytical AI misreads a table, the answer may be wrong.

The platform GroundX.com has a very promising retrieval system built for superior extraction from pdf files. They are part of eyelevel.ai. “APIs for Enterprise-Grade RAG. Built on Kubernetes and fine-tuned open source models. 50% more accurate. Run us in your secure data center.”

Municipal data is inconsistent

Cities do not always use the same categories every year.

A department may be renamed. A fund may be reorganized. A line item may move. A cost may shift from operating to capital. A transfer may appear as both revenue and expense.

The product needs a normalization strategy. Otherwise, the AI may compare things that look similar but are not analytically equivalent.

The system must distinguish facts from derived calculations

The product should clearly separate raw source facts, derived calculations, contextual comparisons, interpretations, and policy judgments.

Layer	Example
Raw source fact	"FY2020 General Fund expenditures were \$42.1 million."
Derived calculation	"The increase from FY2020 to FY2026 was 47%."
Contextual comparison	"That exceeded CPI growth over the same period."
Interpretation	"The increase appears structurally significant."
Policy judgment	"Council should request a recurring-cost explanation."

This distinction is crucial for public trust. The AI should never blur these layers.

Methodology can hide premature conclusions

A premature conclusion can be introduced through:

- period selection,
- peer-city selection,
- inflation measure,
- fund selection,
- employee group selection,
- treatment of vacancies,
- treatment of benefits,
- inclusion or exclusion of one-time costs.

The product should not pretend methodology is neutral. It should show the methodological choices.

Citations must apply to numbers, not just prose

A normal RAG answer cites a paragraph.

An analytical AI must cite:

- the document,
- the page,
- the table,
- the row,
- the column,

- the extracted value,
- the cleaned value,
- the calculation formula.

For funders, this should be understood as a core product requirement, not a nice-to-have.

Human review remains necessary

For marketplace use, the system should include review checkpoints.

Risk level	Example	Review needed
Low	Find a meeting packet	Minimal
Moderate	Summarize staff recommendation	Spot check
High	Calculate budget growth	Validation checks
Very high	Claim that staff evidence was weak	Human analytical review
Very high	Public-facing fiscal risk conclusion	Human review plus methodology disclosure

This is not a weakness. It is part of responsible product design.

The buyer may not be the natural beneficiary

Many civic AI vendors sell to city staff. Their products usually improve service delivery, resident communication, and staff efficiency.

An analytical accountability product benefits:

- council members,
- board members,
- journalists,
- citizens,
- watchdog groups,
- civic researchers,
- candidates,
- advocacy groups,
- internal auditors.

That creates a market challenge:

The entity with the data may not want to buy the tool that makes the data easier to scrutinize.

This supports the case for an independent civic-transparency product outside city infrastructure.

Proposed product architecture

Layer 1: Evidence collection

Purpose: collect and preserve source materials.

Components:

- document scraper,
- public-information file importer,
- PDF archive,
- file hashing,
- metadata registry,
- source URL preservation,
- document-type classifier.

Output:

A trusted public-record library.

Layer 2: Retrieval layer

Purpose: find relevant documents and passages.

Components:

- text extraction,
- chunking,
- embeddings,
- vector database,

- keyword search,
- metadata filters,
- citation labels.

Output:

Source-backed document retrieval.

This is approximately where the current toy product lives.

Layer 3: Structured data layer

Purpose: convert financial and operational records into analysis-ready data.

Components:

- table extraction,
- spreadsheet ingestion,
- data cleaning,
- schema mapping,
- department/fund/account normalization,
- validation checks,
- reconciliation to published totals.

Output:

Trusted municipal data tables.

Layer 4: Analytical tool layer

Purpose: perform calculations outside the language model.

Components:

- SQL query engine,
- R or Python calculation scripts,
- inflation adjustment functions,
- growth-rate functions,
- variance analysis,
- trend analysis,
- control-chart or statistical-process tools where appropriate,
- charting tools,
- data export tools.

Output:

Reproducible calculations and downloadable results.

Layer 5: Argument and assumption layer

Purpose: analyze policy reasoning, not merely numbers.

Components:

- claim extraction,
- recommendation extraction,
- evidence tagging,
- assumption detection,
- rhetorical pattern detection,

- precedent checking,
- longitudinal claim tracking.

Output:

A structured map of municipal decision logic with suggestions for better measures and metrics to assess effectiveness, not just efficiency.

Layer 6: AI answer layer

Purpose: communicate findings clearly.

Components:

- chat interface,
- answer generator,
- citation engine,
- uncertainty warnings,
- calculation explanations,
- chart/table generator,
- downloadable data frames,
- “show your work” panel.

Output:

Public-facing analytical answers with source support.

Layer 7: Governance and quality assurance layer

Purpose: prevent the tool from becoming a confident misinformation machine.

Components:

- audit logs,
- model prompts under version control,
- dataset versioning,
- validation reports,
- human review workflow,
- source confidence scoring,
- red-team testing,
- user feedback loop.

Output:

A product that can be trusted, challenged, and improved.

Suggested funder-facing development stages

Stage 1: Retrieval MVP

Goal:

Demonstrate that users can ask natural-language questions and receive source-backed answers from municipal records.

This has already been largely achieved in toy form.

Deliverables:

- document ingestion,
- chunking,
- embeddings,
- vector search,
- source citations,
- basic Shiny or chat interface.

Primary limitation:

It finds information but does not reliably analyze numbers.

The platform GroundX.com has a very promising retrieval system built for superior extraction from pdf files. They are part of eyelevel.ai. “APIs for Enterprise-Grade RAG. Built on Kubernetes and fine-tuned open source models. 50% more accurate. Run us in your secure data center.”

Stage 2: Structured-data MVP

Goal:

Demonstrate that budget, payroll, and financial tables can be converted into validated data frames.

Deliverables:

- selected budget books parsed,
- selected payroll files imported,
- data dictionary created,
- fiscal-year schema created,
- validation checks built,
- downloadable tables produced.

Primary risk:

PDF table extraction may require document-specific handling.

Stage 3: Analytical MVP

Goal:

Allow the AI to answer limited numerical questions using validated data.

Example questions:

- How much did General Fund spending grow from FY2020 to FY2026?
- Which departments accounted for most of the increase?
- How did compensation growth compare with total expenditure growth?
- What was the annualized growth rate?
- How does the increase compare with inflation?

Deliverables:

- SQL or DuckDB analytical layer,
- calculation functions,
- chart outputs,
- downloadable data,
- calculation citations.

Primary risk:

The system must prevent apples-to-oranges comparisons.

Stage 4: Accountability MVP

Goal:

Link staff claims to evidence, recommendations, decisions, and later outcomes.

Example questions:

- What hard data showing evidence of need was offered for the compensation increase?
- Were peer cities used, and were they justified?
- Did the stated effectiveness problem improve afterward?
- Was this described as one-time or recurring?
- Has the City made the same claim in prior years?

Deliverables:

- claim/evidence extraction,
- assumption tagging,

- longitudinal issue tracking,
- outcome comparison,
- evidence-quality flags.

Primary risk:

This moves from information retrieval into judgment-support, so transparency and human review become essential.

Stage 5: Marketplace pilot

Goal:

Test the product with real users outside the builder's personal workflow.

Potential users:

- council members,
- school board members,
- journalists,
- civic watchdogs,
- resident groups,
- candidates,
- public-finance researchers.

Deliverables:

- hosted application,
- role-based user testing,

- feedback collection,
- onboarding materials,
- sample use cases,
- disclaimers,
- source limitations,
- support model.

Primary risk:

The most natural beneficiaries may not be the government entities that control the documents.

Funder narrative

The following language may be used directly in a concept paper or funding pitch.

- *We have demonstrated the first layer of a civic AI system: retrieval from public municipal records using chunking, embeddings, and source-backed answers. That is useful, but it is not enough for serious public decision support.*
- *The next stage is to build an analytical AI system that can convert public documents and public-information spreadsheets into validated structured data, connect those data to the original records, and allow users to ask questions about financial growth, compensation, budget changes, assumptions, trade-offs, and evidence quality.*
- *The challenge is not merely adding a better chatbot. The hard work is building a trustworthy evidence pipeline: document collection,*

table extraction, data normalization, validation, structured storage, calculation tools, citation tracking, and human-auditable methodology.

- *The resulting product would help citizens and oversight bodies move beyond finding documents toward understanding whether public decisions were supported by evidence, whether claims were consistent over time, and whether promised outcomes actually occurred.*

Blunt summary for funders

The current toy product proves that the front door can work.

A user can ask a question. The system can retrieve relevant chunks. The AI can generate a sourced answer.

But a marketplace-ready analytical AI requires several harder things (some are much harder things):

1. Reliable PDF-to-data extraction
2. Validated financial data models
3. SQL, R, or Python calculation tools
4. Citation trails for every number
5. Longitudinal tracking across fiscal years
6. Claim, evidence, and assumption extraction
7. Methodology transparency
8. Human review for high-risk conclusions
9. A sustainable hosting and update pipeline
10. A buyer/user model that works despite institutional resistance

That is the real funding case:

We are not asking for money to build a chatbot. We are asking for money to build the public evidence infrastructure that allows AI to safely perform civic analysis.

Appendix: Practical implementation notes

Likely transition from local proof of concept to enterprise-ready stack

A practical prototype runs with the existing R-centered workflow, but a marketplace-ready analytical AI product should not be built around R as the production backbone.

R has been extremely useful for exploration, municipal compensation analysis, charting, Quarto reporting, and rapid proof-of-concept work. It should remain part of the project where it is strongest: statistical exploration, analyst notebooks, reproducible reports, and specialized civic-analysis prototypes.

But the enterprise product needs a different spine:

- Python for ingestion, orchestration, extraction, model integration, testing, and production services.
- SQL/PostgreSQL for validated structured data.
- A production vector database such as Pinecone, Qdrant, Weaviate, or pgvector/Postgres for retrieval.
- Cloud object storage for raw PDFs, extracted text, tables, logs, and versioned artifacts.

- API services for the chatbot, analytical tools, user interface, and data export.
- Formal validation, monitoring, security, and deployment workflows.

The core points for funders:

The current R-based workflow proves the analytical concept.

An enterprise product requires a production data platform, software-engineering discipline, and auditable AI infrastructure.

Recommended platform transition

Function	Current proof-of-concept approach	Enterprise-ready direction	Why the transition matters
Document registry	R data frame, CSV, RDS, or DuckDB table	PostgreSQL or managed cloud SQL database	A production registry needs durable IDs, document versioning, audit fields, relational joins, access controls, and reliable back-ups.

Function	Current proof-of-concept approach	Enterprise-ready direction	Why the transition matters
Raw PDF storage	Local files and Cloudflare R2	Cloud object storage such as Cloudflare R2, AWS S3, Azure Blob, or Google Cloud Storage	Raw documents need durable storage, version control, file hashing, lifecycle rules, and repeatable re-processing.
Text extraction	R packages and command-line PDF tools	Python extraction pipeline using tools such as PyMuPDF, pdfplumber, Apache Tika, OCR services, or cloud document AI tools	Enterprise extraction needs scalable batch processing, error handling, page-level outputs, retry logic, and extraction-quality logs.
OCR	Manual or ad hoc	OCR service or containerized OCR workflow	Scanned packets, agenda exhibits, and older PDFs require consistent OCR with confidence scoring

Function	Current proof-of-concept approach	Enterprise-ready direction	Why the transition matters
			and human-review flags.
Table extraction	R/Python experiments with human validation	Python-based table extraction plus validation work-flow; possible use of cloud document AI for hard PDFs	Financial analysis depends on accurate tables. Extraction must preserve row labels, column headers, fiscal years, totals, notes, and source-page references.
Structured analytical storage	RDS, DuckDB, SQLite, or local Postgres	files, PostgreSQL, warehouse layer, or lake-house-style architecture	Analytical data needs schemas, constraints, migrations, repeatable loading, query performance, and multi-user access.
Local analytics engine	R and DuckDB	SQL-first analytics with Python	DuckDB is excellent for portable an-

Function	Current proof-of-concept approach	Enterprise-ready direction	Why the transition matters
		vices; DuckDB retained for analyst sandboxing and fast local testing	analytical work and supports Python and R APIs, but production workflows should have a shared database of record. :contentReference:O
Vector retrieval	Simulated vector store, local embeddings, or R objects	Pinecone, Qdrant, Weaviate, or pgvector/Postgres	Production retrieval needs indexing, metadata filtering, namespaces, scaling, monitoring, and stable APIs. Pinecone supports hybrid retrieval patterns that combine dense and sparse signals; pgvec-

Function	Current of-concept proach	proof- ap- ready direction	Enterprise- Why the transi- tion matters
			tor brings vec- tor search into PostgreSQL with index op- tions such as HNSW and IVFFlat. :con- tentRefer- enceoacite:1
Hybrid search	Basic semantic retrieval	Dense vec- tor search plus keyword/BM25 search plus metadata filter- ing	Civic records often require exact terms, fiscal years, or- dinance num- bers, fund names, ven- dor names, and meet- ing dates. Se- mantic search alone will miss some of that. Pinecone's doc- umentation ex- plicitly recom- mends hybrid

Function	Current proof-of-concept approach	Enterprise-ready direction	Why the transition matters
			approaches because semantic and lexical search have different failure modes. :contentReferenceaioaite:2
Embeddings	Generated locally or experimentally	Scheduled embedding pipeline with model versioning, batching, retries, and re-embedding policy	Enterprise retrieval needs to know which embedding model created each vector and when vectors must be regenerated.
Analytical calculations	cal- R functions and Quarto reports	Tool-called SQL/Python calculation services with tested functions	The language model should not invent calculations. It should call tested tools for growth rates, inflation adjustments, variance

Function	Current proof-of-concept approach	Enterprise-ready direction	Why the transition matters
			analysis, fund comparisons, and downloadable tables.
Interface	Shiny or Quarto dashboard	Web application with API backend; Shiny may remain useful for internal prototypes	Shiny is excellent for analyst-built dashboards, but a public SaaS-style product needs authentication, usage logging, role management, API separation, and deployment practices beyond a typical Shiny app.
Charts and downloads	R-generated charts, CSV, RDS, XLSX	Server-side chart/data export service producing CSV, XLSX, JSON, and possibly	Public users need downloadable results and a visible calculation trail. RDS is useful internally but

Function	Current proof-of-concept approach	Enterprise-ready direction	Why the transition matters
		reproducible notebooks	is not a public-facing interchange format.
AI orchestration	Direct calls from R scripts or Shiny	Python service layer using structured tool calls, retrieval services, SQL services, and response validation	The AI layer should route questions to retrieval, SQL, calculation, or refusal paths rather than letting the model improvise.
Testing	Manual inspection and Quarto debugging	Unit tests, integration tests, golden-answer tests, extraction tests, retrieval tests, and regression tests	A public analytical AI cannot be trusted unless the pipeline is tested after every update.
Deployment	Local machine shinyapps.io	Containerized services using Docker, CI/CD, cloud hosting, secrets management	Enterprise deployment needs repeatability, uptime, controlled releases, rollback,

Function	Current proof-of-concept approach	Enterprise-ready direction	Why the transition matters
		agement, and monitoring	and log visibility.
Governance	Personal view and manual caution	re- Formal data governance, model governance, source limitations, review tiers, audit logs, and user feedback loops	This product will make claims about public finance and governance. It must show its work and preserve an audit trail.

Recommended enterprise architecture

A realistic architecture would separate the system into services rather than treating the chatbot as one large application.

Table 1: Recommended enterprise architecture

Step	Pipeline layer	Purpose
1	Public websites, PDFs, spreadsheets, public records and legally obtained files and FOIA files	Source material collected from official
2	Ingestion service	Downloads, imports, hashes, catalogs, and schedules source updates
3	Raw document storage	Stores original PDFs, spreadsheets, transcripts, and extracted artifacts
4	Document registry database	Maintains document IDs, source URLs, meeting dates, fiscal years, document types, and version history
5	Text extraction, OCR, and table extraction	Converts documents into searchable text and candidate structured tables
6	Validation and reconciliation pipeline	Checks extracted figures against published totals and flags errors for review
7	Structured SQL database	Stores validated budget, payroll, staffing, project, and financial data
8	Vector database / hybrid retrieval index	Supports semantic retrieval, keyword retrieval, and metadata-filtered document search
9	Python API layer	Provides production services for retrieval, SQL queries, calculations, exports, and AI tool calls
10	AI orchestration layer	Routes user questions to search, SQL, calculation tools, citation tools, or refusal paths
11	Web application	Provides the public-facing interface for questions, tables, charts, and downloads
12	Narrative answers, citations, charts, and downloadable data	Delivers user-facing results with visible sources and calculation trails

The important design rules are:

The AI should not be the database, the calculator, or the source of truth.

The AI should be the user-facing reasoning and explanation layer sitting on top of validated data systems.

Role of R going forward

R should not be abandoned, but its role will change.

R remains valuable for:

- analyst notebooks,
- Quarto reports,
- exploratory compensation analysis,
- chart prototypes,
- statistical review,
- one-off civic investigations,
- validating whether a proposed analytical feature is worth building.

R should not be the primary tool for:

- production ingestion services,
- large-scale API development,
- authentication and user management,
- production AI orchestration,

- scalable document processing,
- enterprise deployment,
- multi-tenant application architecture,
- long-term product operations.

A reasonable division of labor is:

Layer	Best-fit tool direction
Exploration and civic analysis	R, Quarto, DuckDB
Production ingestion	Python
PDF/OCR/table extraction	Python and document-processing services
Structured storage	PostgreSQL / cloud SQL
Analytical querying	SQL, Python, possibly DuckDB for sandbox work
Vector retrieval	Pinecone, Qdrant, Weaviate, or pgvector
AI orchestration	Python service layer
Public application	Modern web framework plus API backend
Reporting prototypes	Quarto and R
Enterprise monitoring	Cloud logging, CI/CD, test suites, observability tools

Pinecone, pgvector, and vector-platform caution

Pinecone may be attractive because it is a managed vector database designed for production retrieval workloads. It can support hybrid retrieval patterns, where dense semantic search is combined with sparse or lexical signals. That matters because city records are full of exact

names, fund labels, ordinance numbers, agenda-item IDs, and fiscal-year references that pure semantic search can miss.

However, Pinecone should not be treated as the whole analytical platform.

It stores and retrieves embedded records. It does not solve:

- PDF extraction,
- financial table validation,
- data normalization,
- SQL analysis,
- inflation calculations,
- audit trails,
- claim/evidence mapping,
- human review,
- public-facing methodology disclosure.

pgvector

pgvector is another serious option because it allows vector search inside PostgreSQL. That can simplify architecture by keeping structured municipal data, metadata, and embeddings closer together. The trade-off is that the team becomes responsible for more database tuning, indexing, scaling, and operations. pgvector supports approximate-nearest-neighbor index strategies such as HNSW and IVFFlat, each with performance and recall trade-offs.

The platform decision should not be framed as “Pinecone versus SQL.” A serious analytical civic AI likely needs both:

- SQL for validated facts and calculations.
- Vector search for retrieving relevant text passages.

- Keyword search for exact terms.
- Object storage for source documents.
- AI orchestration to decide which tool to use.
- Suggested first enterprise analytical use case

The first serious analytical use case should remain narrow, but it should be implemented in the new architecture rather than extended only inside the R prototype.

A good candidate would be:

Explain General Fund expenditure growth over a fixed period, identify the departments and categories contributing most to the increase, cite every source number used in the calculation, and provide the user with a downloadable table of the underlying data.

This use case is strong because it forces the product to solve the hard problems:

- collecting official budget documents,
- extracting relevant financial tables,
- preserving page-level citations,
- mapping fiscal years consistently,
- distinguishing actual, adopted, amended, and proposed figures,
- normalizing departments and categories,
- validating totals against published budget summaries,
- loading cleaned figures into SQL,
- calculating dollar growth, percentage growth, annualized growth, and inflation-adjusted comparisons,
- identifying the largest contributors to the increase,
- explaining methodology in plain English,
- producing downloadable CSV/XLSX/JSON outputs,
- preserving an audit trail from answer back to source document.

It is narrow enough to be feasible but serious enough to demonstrate the difference between a toy chatbot and an analytical civic AI product.

Additional cautions for funders

The hardest parts of this project are not the visible chatbot features.

The hard parts are:

Data reliability

- Municipal PDFs were not designed to become databases. The extraction and validation layer will take serious work.

Definition control

- Terms such as “General Fund,” “personnel,” “actual,” “adopted,” “amended,” and “recurring” must be defined consistently.

Version control

- Budgets and staff reports may change. The product must know which version it used.

Citation precision

- Public trust requires citations not only for paragraphs but also for numbers, table cells, formulas, and derived calculations.

Hybrid retrieval

- Semantic search alone is not enough for analyzing civic records. The system must combine vector search, keyword search, metadata filters, and SQL.

Model discipline

- The AI must call tools and report results. It should not perform high-stakes calculations from memory or from loosely retrieved text.

Human in the Loop

- High-risk outputs, especially those criticizing the quality of official evidence, should have review tiers.

Cost control

- Embedding, OCR, vector storage, cloud hosting, monitoring, and LLM calls all create recurring costs.

Institutional resistance

- The users who benefit most may be citizens, journalists, candidates, and oversight officials, not necessarily the municipal staff who control the documents.

Product liability and reputation risk

- A wrong financial conclusion could damage trust quickly. The product must be conservative, auditable, and transparent.

The R prototype can show what is possible. The enterprise product must be rebuilt around production data engineering, SQL-backed analytics, vector retrieval, Python services, validation workflows, and public auditability.